# PV-ASSIST

Rapport complet de projet - Assistant documentaire RAG pour la maintenance photovoltaïque

Projet personnel de Mohamed Douzi | Dossier technique et méthodologique

31 octobre 2026

# Table des matières

**Résumé exécutif** **2**

**1. Contexte, problème et besoin utilisateur** **2**
1.1 Situation de départ . . . 2
1.2 Problème formulé précisément . . . 2
1.3 À qui le projet peut-il être utile ? . . . 2
1.4 Objectifs et critères de réussite . . . 3

**2. Périmètre et choix techniques** **3**
2.1 Fonctions disponibles . . . 3
2.2 Fonctions hors périmètre . . . 3
2.3 Pourquoi ces composants ? . . . 4

**3. Architecture du système** **4**
3.1 Trois environnements à ne pas confondre . . . 5
3.2 Parcours d'indexation . . . 5
3.3 Parcours de question-réponse . . . 5
3.4 Interfaces et responsabilités . . . 6

**4. Détails d'implémentation significatifs** **6**
4.1 Validation et extraction PDF . . . 6
4.2 Chunking et métadonnées . . . 6
4.3 Stockage et recherche Qdrant . . . 6
4.4 Deux responsabilités de l'IA . . . 7
4.5 Abstention et citations . . . 7

**5. Stratégie de tests et assurance qualité** **7**
5.1 Les différents niveaux de test . . . 7
5.2 Scénarios effectivement visibles dans le code de tests communiqué . . . 7
5.3 Exécutions rapportées . . . 8

**6. Évaluation : les chiffres, leur calcul, leur portée** **8**
6.1 Jeu d'évaluation . . . 8
6.2 Que fait le script exactement ? . . . 8
6.3 Résultat observé et formule . . . 8
6.4 Risques statistiques et méthodologiques . . . 9
6.5 Étapes suivantes pour évaluer l'ensemble du système . . . 9

**7. Sécurité, fiabilité et limites** **9**

**8. Reproductibilité et mode opératoire** **9**

**9. Améliorations proposées et priorisation** **10**

**10. Bilan critique et synthèse pour recruteur** **10**

**Annexe A. Glossaire condensé** **11**

**Annexe B. Traçabilité et prudence méthodologique** **11**

# Résumé exécutif

**PV-Assist** est un prototype local d'assistant de recherche et de question-réponse sur des manuels PDF de maintenance photovoltaïque. Il vise un problème concret : les consignes utiles à un technicien peuvent être dispersées dans plusieurs pages de documentation ; les retrouver rapidement, sans confondre connaissances générales et prescriptions propres à un équipement, est difficile. L'application permet d'importer un manuel, de poser une question en langage naturel et de consulter une réponse accompagnée de passages et de numéros de page.

Le principe technique est le **RAG, Retrieval-Augmented Generation** : une recherche documentaire intervient avant la génération de texte. Les fichiers PDF sont extraits avec PyMuPDF, découpés en passages, représentés par des vecteurs locaux calculés avec FastEmbed puis stockés dans Qdrant. Lors d'une question, le système récupère les passages les plus proches sur le plan sémantique ; un modèle génératif accessible par API rédige une réponse contrainte par ces passages. FastAPI expose le service, Streamlit fournit l'interface, et Qdrant s'exécute dans un conteneur Docker.

Le prototype comporte des garde-fous : filtrage des passages selon un score minimal, abstention si les informations sont insuffisantes, identifiants de sources `[S1]` et validation de l'existence de ces identifiants avant affichage. La validation de citation reste **structurelle** : elle ne prouve pas que chaque phrase générée est réellement soutenue par le texte cité.

**Résultats attestés par les sorties et fichiers communiqués :** 30 tests automatisés réussis ; contrôle Ruff réussi ; essai réel de réponse pertinente via Groq ; abstention observée pour une question portant sur une tension DC maximale absente du manuel ; benchmark hors ligne de 15 questions, dont 11 répondables et 4 appelant une abstention. Pour les 11 questions répondables, la page attendue apparaît dans les deux premiers résultats de recherche, soit **Page Hit Rate@2 = 11/11 = 100 %** sur un **manuel synthétique de trois pages**. Ce chiffre mesure la récupération documentaire, **pas** l'exactitude des réponses du LLM. Le benchmark ne déclenche aucun appel LLM et n'a contrôlé aucune citation faute de fichier de réponses enregistrées.

**Statut :** démonstrateur local fonctionnel et testé sur un petit corpus ; **non validé pour guider des opérations électriques réelles et non prêt pour la production**.

# 1. Contexte, problème et besoin utilisateur

## 1.1 Situation de départ

Les installations photovoltaïques réunissent des modules, onduleurs, raccordements, organes de protection et procédures spécifiques aux fabricants et aux sites. Leur entretien s'appuie sur de la documentation potentiellement longue. Une question ponctuelle - par exemple « Quel matériau utiliser pour nettoyer les modules ? » - oblige à retrouver la bonne section, vérifier qu'elle concerne l'équipement et lire les avertissements associés.

Les moteurs de recherche textuelle classiques reposent souvent sur la présence des mêmes mots : une question formulée en « nettoyage des panneaux » peut devoir retrouver une phrase formulée en « cleaning photovoltaic modules ». Les modèles génératifs généralistes, de leur côté, produisent un texte plausible sans nécessairement connaître le manuel du site. Dans les deux cas, l'utilisateur doit vérifier les informations critiques.

## 1.2 Problème formulé précisément

> **Comment permettre à un utilisateur de retrouver plus facilement des informations contenues dans un manuel photovoltaïque donné, tout en rendant visible la provenance des réponses et en évitant de présenter des prescriptions absentes de la documentation comme des faits établis ?**

Il ne s'agit **pas** de diagnostiquer automatiquement une installation, de piloter un équipement, ni de remplacer les procédures approuvées ou les compétences d'un personnel habilité.

## 1.3 À qui le projet peut-il être utile ?

| Public potentiel | Besoin | Bénéfice visé | Limite impérative |
|---|---|---|---|
| Technicien de maintenance habilité | Retrouver rapidement une consigne documentée | Accès à la page et au passage du manuel | Relecture de la documentation et des procédures de sécurité |
| Responsable d'exploitation | Consulter différentes documentations techniques | Recherche contextualisée par document | Ne remplace pas un processus de validation |
| Formateur ou étudiant | Explorer un manuel et confronter une réponse à sa source | Apprentissage et navigation documentaire | Corpus d'essai synthétique, pas une procédure terrain |
| Équipe support technique | Orienter la recherche dans des notices de produits | Repérage plus rapide des extraits pertinents | Recommandations à confirmer par experts et documents officiels |

L'utilité est **plausible**, mais le gain de temps et la satisfaction utilisateur **n'ont pas été mesurés** expérimentalement dans ce projet.

### 1.4 Objectifs et critères de réussite

Objectifs fonctionnels : importer un PDF, extraire le texte, indexer les passages, chercher dans un document donné, poser une question, montrer la source et refuser une réponse insuffisamment documentée. Objectifs techniques : séparer interface et backend, isoler la base vectorielle, permettre deux fournisseurs LLM, automatiser une part importante des tests, disposer d'une évaluation reproductible.

Un MVP réussi démontre que le parcours fonctionne sur une documentation de test. Une version de production exigerait, en plus, de la sécurité, des évaluations sur de vrais corpus, une surveillance des erreurs et un processus de validation métier.

## 2. Périmètre et choix techniques

### 2.1 Fonctions disponibles

- Chargement d'un PDF ; validation du contenu ; extraction du texte page par page ; découpage des pages en chunks d'environ 700 caractères avec 150 caractères de chevauchement.
- Identification du document à partir d'un SHA-256 du contenu binaire ; identification stable des chunks et points ; remplacement des anciens passages lors d'une nouvelle indexation portant le même nom de document.
- Calcul d'embeddings local avec `BAAI/bge-small-en-v1.5` (384 dimensions), stockage dans Qdrant et recherche vectorielle avec distance cosinus.
- Recherche optionnellement limitée à un `document_id`.
- Backend FastAPI proposant `/health`, `/documents/upload`, `/search`, `/ask` ; interface Streamlit pour importer, interroger et afficher sources et passages.
- Génération via Groq (`openai/gpt-oss-20b` dans les essais) ou fournisseur Mistral alternatif (`mistral-small-latest` configuré) ; contrôle des identifiants de sources et mécanisme d'abstention.
- Tests avec pytest, doubles de test des services externes, Qdrant réel pour un test d'intégration, AppTest de Streamlit et évaluation hors ligne.

### 2.2 Fonctions hors périmètre

Aucune reconnaissance OCR des documents scannés n'est annoncée ; aucune recherche hybride lexicale/vectorielle ni reranking, aucun vérificateur sémantique de phrases, aucun contrôle d'accès multi-utilisateur, aucun déploiement cloud ni supervision de production. Le système n'est pas un outil de contrôle de sécurité électrique.

### 2.3 Pourquoi ces composants ?

| Composant | Rôle | Motivation et compromis |
|---|---|---|
| Python 3.12 | Langage du projet | Écosystème ML/API ; dépendances à installer |
| PyMuPDF | Extraction PDF | Accès au texte et aux pages ; pas d'OCR intégré au pipeline |
| FastEmbed | Modèle d'embeddings | Exécution locale et simple ; qualité dépendante du modèle/corpus |
| Qdrant | Base de vecteurs | Recherche cosinus, métadonnées et filtres ; service additionnel à opérer |
| Docker Compose | Exécution de Qdrant | Démarrage reproductible ; n'héberge pas tout le projet |
| FastAPI + Pydantic | Backend HTTP typé | Routes testables ; nécessite gestion des erreurs et réseau |
| Streamlit | Interface | Prototype rapide ; personnalisations UX limitées |
| Groq / Mistral API | Génération | Modèles servis à distance ; clé, quota et réseau nécessaires |
| pytest / Ruff | Contrôle qualité | Régression et lint ; ne certifient pas la justesse métier |

## 3. Architecture du système




Figure 1: Pipeline d'indexation

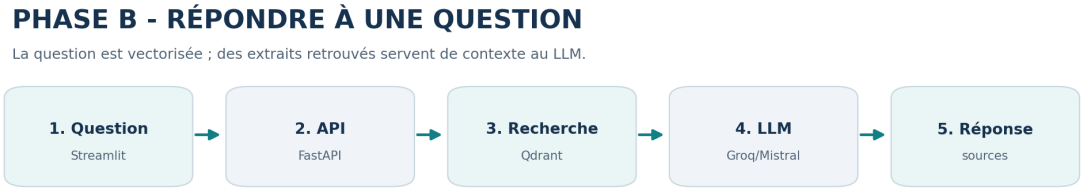



Figure 2: Pipeline de questions




Figure 3: Architecture de communication

## 3.1 Trois environnements à ne pas confondre

**Sur le PC Windows :** l'environnement Python `.venv`, FastAPI sur le port 8001, Streamlit sur le port 8501 et le modèle d'embeddings FastEmbed exécuté par l'application Python. **Dans Docker sur ce même PC :** le serveur Qdrant, accessible typiquement via le port 6333. **Hors de ce PC :** l'API du fournisseur d'inférence Groq ou Mistral, contactée en HTTPS à la génération. Le PDF et les embeddings restent locaux dans le pipeline décrit ; des extraits retrouvés sont transmis au fournisseur LLM lors d'une question.

La formule « Qdrant est conteneurisé » signifie seulement que **le serveur de base vectorielle s'exécute dans un conteneur Docker**. Elle ne signifie pas que l'ensemble de l'application est conteneurisé ni déployé publiquement.

## 3.2 Parcours d'indexation

1. Le navigateur charge un PDF dans Streamlit. Streamlit envoie une requête HTTP à `POST /documents/upload` sur FastAPI.
2. FastAPI valide le fichier et appelle `DocumentIndex.ingest_document`.
3. PyMuPDF lit le contenu des pages ; le système coupe chaque page en chunks, qui conservent la référence de leur page.
4. FastEmbed calcule un vecteur numérique de 384 valeurs par chunk.
5. Qdrant reçoit les points `(id, vector, payload)` ; `payload` contient nom du manuel, identifiant, page et texte.
6. L'API renvoie le `document_id` SHA-256 et le nombre de chunks ; Streamlit sait désormais quel manuel interroger.

**Conséquence :** l'indexation est réalisée lors du chargement, et non nécessairement à chaque question ; cela évite de recalculer les embeddings du document pour chaque requête.

## 3.3 Parcours de question-réponse

1. Un utilisateur saisit une question dans Streamlit et confirme l'envoi.
2. Streamlit transmet un JSON à `POST /ask` avec le champ `question` et éventuellement `document_id`.
3. FastEmbed convertit la question en vecteur 384D ; Qdrant compare ce vecteur à ceux des chunks déjà stockés.
4. Les résultats sont ordonnés par similarité ; les passages sous `min_retrieval_score` (valeur discutée : 0,35) sont ignorés.
5. Les passages conservés deviennent les sources `[S1]`, `[S2]`, etc. Si aucune source ne reste, l'API renvoie une abstention sans appeler le LLM.
6. Si des sources existent, l'API construit un contexte avec question et extraits, puis appelle le fournisseur LLM.
7. Le backend vérifie que les identifiants cités sont présents parmi les sources autorisées ; les citations absentes ou inconnues sont refusées, sauf format prévu d'abstention.
8. Le JSON de réponse contient texte, sources et indicateur `insufficient_context`. Streamlit affiche des références de pages lisibles et les passages que l'on peut ouvrir.

### 3.4 Interfaces et responsabilités

- **`config.py` :** lecture de paramètres, adresses, modèle, clés et seuils ; secrets conservés dans `.env` local non versionné.
- **`models.py` :** contrats Pydantic des chunks, résultats de recherche, sources et requêtes/réponses.
- **`services.py` :** opérations documentaires et vectorielles avec `EmbeddingService`, `QdrantVectorStore`, `DocumentIndex`.
- **`generation.py` :** services Groq/Mistral, assemblage des sources, instructions de génération, erreurs fournisseur et validation structurelle de citations.
- **`api.py` :** orchestration HTTP, codes de statut et gestion des scénarios sans preuve.
- **`ui.py` :** état d'interface, chargement, boutons, affichage des citations et passages.
- **`scripts/evaluate_rag.py` :** benchmark de retrieval et contrôle optionnel d'identifiants dans des réponses enregistrées ; ne déclenche pas d'appel au modèle.

## 4. Détails d'implémentation significatifs

### 4.1 Validation et extraction PDF

L'application examine les octets du PDF, pas seulement son extension ; elle refuse un fichier vide, trop volumineux, invalide ou sans page. Une extraction simplifiée ressemble à :

```
with fitz.open(stream=file_bytes, filetype="pdf") as doc:
    for page_number in range(doc.page_count):
        text = doc.load_page(page_number).get_text("text", sort=True)
        pages.append({"page_number": page_number + 1, "text": text.strip()})
```

*Extrait explicatif simplifié :* dans l'implémentation transmise, la fermeture est assurée par `try/finally`. Les pages sont numérotées à partir de 1, car c'est ainsi que l'utilisateur les consulte.

### 4.2 Chunking et métadonnées

Le découpage s'effectue **page par page** avec `chunk_size=700` et `chunk_overlap=150` dans les paramètres examinés. Exemple sur 1 400 caractères : indices de départ 0, 550 et 1 100, soit trois chunks. La redondance limite les coupures de contexte, mais augmente le stockage.

Chaque chunk porte `chunk_id`, `point_id`, `document_id`, `document_name`, `page_number` et `text`. Le SHA-256 identifie les **octets du document** ; l'UUID5 sert à construire un identifiant Qdrant déterministe à partir de l'identité et de la position du chunk. Ces empreintes ne constituent pas du chiffrement.

### 4.3 Stockage et recherche Qdrant

Un point typique :

```
{
  "id": chunk.point_id,
  "vector": embedding,       # 384 nombres flottants
  "payload": {
    "document_id": chunk.document_id,
    "document_name": chunk.document_name,
    "page_number": chunk.page_number,
    "chunk_id": chunk.chunk_id,
    "text": chunk.text
  }
}
```

La collection est définie pour des vecteurs de dimension compatible et une comparaison cosinus. en-

`sure_collection` signale une erreur si la taille attendue ne correspond pas à celle d'une collection existante. La recherche peut appliquer un filtre `document_id` côté Qdrant. Dans le code transmis, le remplacement par nom de document effectue un `upsert` des points courants puis supprime ceux portant le même nom mais plus les mêmes identifiants. Cette simplicité devient un risque de collision dans une future application multi-utilisateur.

### 4.4 Deux responsabilités de l'IA

- **Embedding model :** `BAAI/bge-small-en-v1.5` représente des passages et questions en vecteurs. Il n'écrit pas la réponse et fonctionne localement.
- **Generative LLM :** le fournisseur Groq héberge `openai/gpt-oss-20b` pour l'essai validé ; l'adaptateur Mistral peut appeler `mistral-small-latest` sous réserve de quota. Ce modèle reçoit la question plus les passages, puis génère une réponse en langue naturelle. Les poids ne sont pas réentraînés.

### 4.5 Abstention et citations

Le système possède deux occasions de s'abstenir : **absence de passage suffisamment similaire**, puis **décision explicite du LLM** lorsqu'une question reste non résolue malgré des passages proches. L'identifiant `[S1]` relie une phrase générée à un passage connu. L'API contrôle la présence de cet identifiant ; elle **ne calcule pas** si la phrase est logiquement impliquée par le passage. L'exemple « le maximum est 1 000 V [S1] » peut donc passer le contrôle de format même si S1 ne donne aucun maximum : c'est un risque résiduel critique.

## 5. Stratégie de tests et assurance qualité

### 5.1 Les différents niveaux de test

**Test unitaire :** vérifie une fonction ou une règle isolée. Exemple : refuser des paramètres de découpage incohérents, ou contrôler un identifiant de citation. **Test avec mocks :** substitue un faux client Groq/Mistral afin de vérifier prompt, gestion des erreurs, absence de réessai, sélection du fournisseur, sans dépendre de la facturation. **Test d'API :** utilise `TestClient` pour envoyer un faux `POST /ask` et contrôler le statut et le JSON. **Test d'intégration :** implique Qdrant réel en conteneur pour vérifier le stockage et la récupération. **Test d'interface :** Streamlit `AppTest` teste la logique des boutons et la présentation des sources. **Essai manuel réel :** interroge un vrai fournisseur LLM et peut donc être variable et consommer un quota.

### 5.2 Scénarios effectivement visibles dans le code de tests communiqué

| Domaine | Exemple de scénario vérifié | Ce que cela prouve |
|---|---|---|
| Fournisseur Groq | Envoi d'un prompt fondé sur des sources, analyse de la réponse | Construction correcte de la requête simulée |
| Fournisseur Mistral | Construction d'une requête avec extraits | Correcte interaction avec un faux client |
| Configuration | Sélection du fournisseur, absence de clé lorsqu'aucune génération | Bonne logique de sélection et chargement différé des clés |
| Gestion 429 / transport | Erreur sûre, non divulguée et sans réessai automatique | Comportement logiciel sous erreur simulée |
| `/ask` | Réponse avec `[S1]`, page, passage et `document_id` | Contrat JSON sur doubles de test |
| `/ask` sans preuve | 0 résultat ou score sous seuil : abstention sans génération | Économie d'appels et évitement d'une réponse hors contexte |
| `/ask` citation erronée | Réponse sans citation ou `[S99]` inconnue rejetée | Validation structurelle des identifiants |
| `/ask` abstention | Même avec score 0,99, abstention explicite acceptée | Score élevé n'impose pas une réponse |

| Domaine | Exemple de scénario vérifié | Ce que cela prouve |
|---|---|---|
| Streamlit | Le formulaire reste désactivé avant indexation ; ancien résultat effacé au changement de document | Logique d'état et d'affichage de l'interface |
| Qdrant réel | Test d'intégration dans la suite annoncé et exécuté | Interaction entre application et base vectorielle locale |

Le fichier de tests transmis contient plusieurs scénarios paramétrés. Le total **30 tests réussis** provient du journal pytest communiqué ; sans extraction de la liste complète du dépôt distant, il serait imprudent d'inventer trente noms de tests. Les descriptions ci-dessus s'appuient sur le code visible et le compte total communiqué.

### 5.3 Exécutions rapportées

```
pytest: 30 passed in 10.75s
ruff:   All checks passed!
```

Un test passé signifie que **ses assertions ont réussi dans l'environnement du test**. Il ne signifie pas que toutes les conditions d'utilisation ont été envisagées. Un mock Groq ne teste pas la qualité des réponses de Groq en conditions réelles ; un test Qdrant réel ne prouve pas le comportement sur des millions de documents.

## 6. Évaluation : les chiffres, leur calcul, leur portée

### 6.1 Jeu d'évaluation

Le fichier `evaluation/rag_questions.json` comporte **15 questions**, construites autour d'un **manuel synthétique de trois pages** : 11 questions dont une page de référence est indiquée et 4 questions pour lesquelles une abstention est attendue. Parmi les informations volontairement absentes : une procédure détaillée de consignation, une marque et dilution de détergent, une différence maximale de température entre modules, et une classe d'EPI avec distance d'arc électrique et norme applicable.

### 6.2 Que fait le script exactement ?

`python scripts/evaluate_rag.py --top-k 2` lit le jeu de questions, calcule l'identifiant SHA-256 du manuel, cherche dans Qdrant pour chaque question, rassemble les pages retournées, compare les pages obtenues aux pages attendues, puis imprime un rapport JSON. Il emploie le **modèle d'embeddings** pour calculer les vecteurs de questions et effectue de vraies recherches dans Qdrant ; il n'appelle **pas** le modèle génératif Groq/Mistral.

Le drapeau `--answers-json`, s'il reçoit un objet JSON d'anciennes réponses déjà générées, ajoute un contrôle **des identifiants de sources**. Lors de l'exécution présentée, ce fichier n'était pas fourni ; **zéro réponse** a donc été soumise à cette vérification.

### 6.3 Résultat observé et formule

$$\text{PageHitRate@2} = \frac{\text{questions répondables ayant une page attendue dans le top 2}}{\text{nombre de questions répondables}} = \frac{11}{11} = 1 = 100\%.$$

Cela vérifie **une étape intermédiaire** : le récupérateur a localisé une page utile parmi les deux premiers passages des questions annotées. Cela ne mesure ni la qualité du raisonnement du LLM, ni la conformité de chaque citation à une phrase, ni le taux de bonnes abstentions. Les quatre questions sans réponse ne rentrent pas dans le dénominateur de cette métrique.

**Pourquoi « 0 appel LLM » ?** Parce que le script est volontairement un **test de recherche documentaire sans**

**génération** : il interroge FastEmbed + Qdrant, pas Groq/Mistral. **Pourquoi « 0 citation évaluée » ?** Parce qu'il n'avait **aucune réponse générée enregistrée** à examiner. Ces deux zéros ne veulent ni dire que l'application ne sait pas appeler un LLM, ni que les citations sont toutes fausses ; ils décrivent les limites de **cette exécution d'évaluation**.

### 6.4 Risques statistiques et méthodologiques

11 questions et trois pages constituent une population d'essai très petite. Avec k=2, récupérer l'une de trois pages attendues peut être relativement facile ; de plus, plusieurs chunks peuvent appartenir à une même page. Le jeu de développement peut ressembler fortement au manuel utilisé pour le construire. Aucune incertitude statistique ni généralisation à de vrais documents n'a été établie. La phrase « 100 % accuracy du RAG » serait scientifiquement incorrecte.

### 6.5 Étapes suivantes pour évaluer l'ensemble du système

Établir un corpus autorisé de vrais manuels variés ; définir des questions avec **réponse de référence**, preuves exactes et absence d'information ; figer la version de l'index, du modèle et des paramètres ; collecter des réponses générées ; demander à des évaluateurs humains de juger **exactitude**, **support des citations**, **abstention appropriée** et **sécurité** ; rapporter les échecs par catégorie et mesurer la latence/coût. Prévoir un test de non-régression sur les cas de sécurité électrique.

## 7. Sécurité, fiabilité et limites

**Sécurité des connaissances :** le PDF est une source documentaire externe ; le texte peut comporter des instructions malveillantes (*prompt injection*) et ne doit pas avoir l'autorité d'un message système. Les extraits peuvent être partagés avec le fournisseur LLM lors de la génération ; les documents sensibles nécessitent un contrôle de confidentialité et de traitement des données.

**Sécurité applicative :** `.env` ne doit pas être publié ; la clé API ne doit pas apparaître dans les logs. Le prototype ne dispose pas des éléments d'une plateforme multi-utilisateur sécurisée : comptes, autorisations par document, isolation de locataires, limitation de débit, journalisation maîtrisée et politique de rétention.

**Sécurité métier :** l'application ne doit pas inventer de tension, d'ordre de consignation, de distance de sécurité ou de mesure réglementaire. L'abstention est un mécanisme de réduction de risque ; elle n'est pas une garantie. Seuls les documents et procédures officiellement applicables et les personnes habilitées font autorité pour des interventions.

**Limites documentaires :** PDF scannés sans OCR, structure complexe de tableaux, données sur deux pages, schémas électriques, versions de manuels et multilinguisme. Un vecteur de texte seul n'interprète pas nécessairement correctement les éléments visuels.

**Limites de validation :** absence de mesure de factualité sémantique, de benchmarking de latence, de coût, de charge et d'utilité utilisateur. Les services de génération dépendent de fournisseurs distants et de quotas.

## 8. Reproductibilité et mode opératoire

Depuis la racine du dépôt sur Windows PowerShell :

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

docker compose up -d qdrant
if (!(Test-Path .env)) { Copy-Item .env.example .env }
notepad .env                    # renseigner localement LLM_PROVIDER et la clé
```

Dans un premier terminal :

```
$env:PYTHONPATH = "src"
.\.venv\Scripts\python.exe -m uvicorn pv_assist.api:app --host 127.0.0.1 --port 8001
```

Dans un deuxième terminal :

```
$env:PYTHONPATH = "src"
$env:PV_ASSIST_API_URL = "http://127.0.0.1:8001"
.\.venv\Scripts\python.exe -m streamlit run src/pv_assist/ui.py
```

Ouvrir `http://localhost:8501`, indexer le PDF d'essai, poser une question, développer les passages de référence. L'interface `/docs` de FastAPI se trouve à `http://localhost:8001/docs`.

Tests et benchmark :

```
.\.venv\Scripts\python.exe -m pytest -v
.\.venv\Scripts\python.exe -m ruff check .
.\.venv\Scripts\python.exe scripts/evaluate_rag.py --top-k 2
```

**Précondition :** Qdrant doit fonctionner, et le PDF `tests/fixtures/pv_maintenance_smoke.pdf` doit avoir été indexé avant le benchmark. Le benchmark utilise FastEmbed/Qdrant, **pas de clé LLM** ; la génération réelle nécessite une clé Groq ou Mistral valide.

## 9. Améliorations proposées et priorisation

| Priorité | Travail | Raison |
|---|---|---|
| P0 | Vérification humaine et juridique des documents utilisés | Éviter l'usage de manuels inappropriés pour des conseils sensibles |
| P0 | Benchmark de réponses et d'abstention sur cas réalistes | Mesurer enfin l'objectif de fiabilité |
| P1 | Gestion sûre des utilisateurs, droits et documents | Éviter la fuite et la collision entre utilisateurs |
| P1 | OCR, tableaux, sections et pages multiples | Étendre la couverture documentaire |
| P1 | Évaluation des preuves au niveau de la phrase | Aller au-delà de la validation structurelle [S1] |
| P2 | Recherche hybride et reranking | Améliorer les cas de termes techniques et références exactes |
| P2 | Journalisation, métriques, coût, latence et tests de charge | Qualifier une exploitation réelle |
| P2 | Packaging/containers pour l'ensemble des services | Faciliter la livraison du produit |

## 10. Bilan critique et synthèse pour recruteur

Le principal apport de PV-Assist est la **construction d'un pipeline transversal** : document PDF, traitement du texte, embeddings, index vectoriel, API, interface, fournisseur LLM, citations et tests. Le projet illustre aussi une démarche de rigueur : séparer **recherche documentaire** et **génération**, concevoir des scénarios d'abstention, et communiquer les limites du résultat 11/11.

L'expérience n'autorise pas les affirmations « système de maintenance certifié », « 100 % de réponses exactes », « déploiement cloud » ou « application entière dans Docker ». L'usage d'un assistant de code fait partie du processus de développement ; la compétence à démontrer en entretien est la compréhension des composants, des choix, des essais, du débogage et des risques.

**Présentation courte possible :**

> PV-Assist est un assistant documentaire RAG pour la maintenance photovoltaïque. Il extrait et découpe des PDF, encode les passages en vecteurs locaux, les indexe dans Qdrant via Docker et les récupère lors d'une question. FastAPI orchestre la recherche et la génération avec un LLM ; Streamlit présente la réponse avec pages et sources. J'ai ajouté des abstentions, des contrôles structurels de citations, des tests automatisés et un benchmark de récupération. Sur un petit manuel synthétique, 11/11 questions répondables ont retrouvé leur page dans le top 2 ; la justesse factuelle des réponses reste à évaluer séparément.

## Annexe A. Glossaire condensé

| Terme | Définition dans ce projet |
|---|---|
| API | Interface HTTP par laquelle Streamlit demande du travail à FastAPI |
| Endpoint | Adresse d'une opération HTTP, par exemple `POST /ask` |
| RAG | Recherche d'extraits pertinents **avant** génération de la réponse |
| Chunk | Petit passage découpé dans une page PDF |
| Embedding | Vecteur numérique produit par un modèle pour représenter un texte |
| Qdrant | Base qui stocke ces vecteurs et retrouve leurs voisins |
| Docker | Outil pour lancer Qdrant dans un conteneur isolé |
| LLM | Modèle de langage qui rédige la réponse à partir des sources |
| Groq | Fournisseur d'inférence ayant servi aux essais réels réussis |
| Citation `[S1]` | Identifiant temporaire renvoyant à un passage récupéré |
| Mock | Faux composant remplaçant un service réel dans un test |
| Page Hit Rate@2 | Part des questions répondables retrouvant une page attendue dans les deux premiers résultats |

## Annexe B. Traçabilité et prudence méthodologique

Ce rapport est établi à partir des extraits `services.py`, du fichier de tests communiqué, du README et du script d'évaluation que l'utilisateur a copiés, des logs d'exécution fournis et du cours technique antérieur. Il ne repose **pas** sur un audit exhaustif du dépôt GitHub dans son état actuel. Les extraits de code présentés comme « simplifiés » servent à expliquer la logique et ne sont pas des reproductions contractuelles ligne par ligne. Les chiffres sont des **résultats rapportés** et non une nouvelle exécution dans cet environnement. Date du document : 31 octobre 2026.